

M Y O U - D D - 0 0 1

ミュー株式会社 御中

製品トレーサビリティ・有効期限管理システム

詳細設計書

対象システム：製品トレーサビリティ機能

対象製品：セルフィール MS（スプレー缶）

文書番号	MYOU-DD-001
版数	第 1.0 版
発行日	2026年7月29日

改訂履歴

版数	改訂日	改訂内容	改訂者
1.0	2026年7月29日	初版発行	—

目次

1. はじめに

- 1.1 本書の目的
- 1.2 表記規則

2. モジュール構成

- 2.1 ディレクトリ構成
- 2.2 モジュール責務

3. データベース詳細設計

- 3.1 myou_companies（施工会社）
- 3.2 myou_lots（製造ロット）
- 3.3 myou_delivery_logs（出荷履歴）
- 3.4 myou_trace_labels（トレーサビリティQR 発行）
- 3.5 myou_alert_logs（アラート送信履歴）

4. データベース関数（RPC）詳細設計

- 4.1 myou_receive_lot（入荷登録）
- 4.2 myou_deliver_from_lot（出荷登録）

5. サーバー処理詳細設計

- 5.1 共通事項
- 5.2 入力検証スキーマ
- 5.3 更新処理（Server Actions）
- 5.4 読み取り処理（Queries）

6. 共通ロジック詳細設計

- 6.1 QR 解析・採番（`lib/qr-parser.ts`）
- 6.2 有効期限計算（`lib/expiration.ts`）
- 6.3 処理ステータス（`lib/process-status.ts`）

7. 画面詳細設計

- 7.1 入荷登録（`/myou/receiving-scan`）
- 7.2 在庫一覧（`/myou/inventory`）
- 7.3 出荷登録（`/myou/delivery-scan`）
- 7.4 出荷リスト（`/myou/delivery-history`）
- 7.5 トレーサビリティ検索（`/myou/traceability`）
- 7.6 有効期限監視・アラート（`/myou/expiration-alerts`）
- 7.7 納入先登録（`/myou/companies`）
- 7.8 製品情報公開ページ（`/p/myou/trace/{id}`）

8. メール送信詳細設計

9. エラーハンドリング設計

9.1 方針

9.2 エラー区分

10. 単体テスト

11. 制限値一覧

12. 環境変数

1. はじめに

1.1 本書の目的

本書は、基本設計書（MYOU-BD-001）で定義した方式に基づき、実装レベルの仕様（モジュール構成、テーブル定義、処理仕様、メッセージ、制限値）を定義したものである。保守・改修の際の一次資料として使用する。

1.2 表記規則

表記	意味
モノスペース	ファイル名、関数名、テーブル名、列名、コード値
型「文字列」	PostgreSQL の text 型
型「日付」	PostgreSQL の date 型
型「日時」	PostgreSQL の timestampz 型

2. モジュール構成

2.1 ディレクトリ構成

src/	
├── app/(tenant)/(tenant-users)/myou/	… 画面（ルーティング）
│ ├── receiving-scan/page.tsx	… 入荷登録
│ ├── inventory/page.tsx	… 在庫一覧
│ ├── delivery-scan/page.tsx	… 出荷登録
│ ├── delivery-history/page.tsx	… 出荷リスト
│ ├── traceability/page.tsx	… トレーサビリティ検索
│ ├── expiration-alerts/page.tsx	… 有効期限監視・アラート
│ ├── companies/page.tsx	… 納入先登録
│ ├── doc-index/page.tsx	… 資料一覧
│ ├── manual/page.tsx	… システム仕様書
│ ├── loading.tsx / error.tsx	… 読込中・エラー画面
│ └── components/	… 画面構成部品（29ファイル）
├── app/p/myou/	… 公開ページ（認証不要）
│ ├── trace/[id]/page.tsx	… 製品情報公開ページ
│ └── product-manuals/page.tsx	… 製品マニュアル公開ページ
├── features/myou/	… 業務ロジック
│ ├── queries.ts	… 読み取り処理
│ ├── actions.ts	… 更新処理（Server Actions）
│ ├── types.ts	… 型定義・入力検証スキーマ
│ ├── lib/qr-parser.ts	… QR 解析・採番
│ ├── lib/expiration.ts	… 有効期限の残日数計算
│ ├── lib/process-status.ts	… 処理ステータス
│ └── manual/	… システム仕様書の読込・目次生成
└── supabase/migrations/	… データベース定義（SQL）

2.2 モジュール責務

モジュール	責務	制約
page.tsx	画面の組み立てとデータ取得の起点	データベースへ直接アクセスしない
queries.ts	読み取り（SELECT）専用	更新処理を含めない
actions.ts	更新（INSERT / UPDATE / DELETE）	管理者用クライアント（RLS バイパス）を使用しない
lib/*.ts	副作用のない純粋関数	データベース・ネットワークへアクセスしない。単体テスト対象
components/*.tsx	画面表示と操作	更新は actions.ts の関数呼び出しに限る

3. データベース詳細設計

3.1 myou_companies（施工会社）

列名	型	NULL	既定値	制約・備考
id	UUID	不可	gen_random_uuid()	主キー
tenant_id	UUID	不可	current_tenant_id()	テナント識別子
name	文字列	不可	—	会社名
email_address	文字列	可	—	アラート送信先。未登録は NULL
company_no	整数	不可	—	出荷先No。(tenant_id, company_no) で一意
created_at	日時	不可	now()	登録日時

インデックス： idx_myou_companies_tenant (tenant_id)

RLS： Tenant Isolation for myou_companies — FOR ALL TO authenticated USING (tenant_id = current_tenant_id())

3.2 myou_lots（製造ロット）

列名	型	NULL	既定値	制約・備考
id	UUID	不可	gen_random_uuid()	主キー
tenant_id	UUID	不可	current_tenant_id()	テナント識別子
lot_no	文字列	不可	—	(tenant_id, lot_no) で一意
qr_payload	文字列	不可	—	ロットQRの内容
manufactured_date	日付	可	—	製造日
expiration_date	日付	不可	—	有効期限
quantity_total	整数	不可	0	CHECK (quantity_total >= 0)
quantity_remaining	整数	不可	0	CHECK (quantity_remaining >= 0)
status	文字列	不可	'issued'	CHECK (status IN ('issued', 'in_stock', 'depleted'))
received_at	日付	可	—	入荷日
created_at	日時	不可	now()	登録日時

テーブル制約： myou_lots_remaining_lte_total — CHECK (quantity_remaining <= quantity_total)

インデックス： idx_myou_lots_tenant_status_exp (tenant_id, status, expiration_date)

RLS： Tenant Isolation for myou_lots

残数が総数を超えない制約をデータベース側に持たせることで、アプリケーションの不具合による在庫の不整合を防いでいる。

3.3 myou_delivery_logs（出荷履歴）

列名	型	NULL	既定値	制約・備考
id	UUID	不可	gen_random_uuid()	主キー
tenant_id	UUID	不可	current_tenant_id()	テナント識別子
lot_id	UUID	不可	—	外部キー → myou_lots(id)
company_id	UUID	不可	—	外部キー → myou_companies(id)
quantity	整数	不可	1	CHECK (quantity > 0)
used_quantity	整数	不可	0	CHECK (used_quantity >= 0 AND used_quantity <= quantity)
delivery_date	日付	不可	CURRENT_DATE	出荷日
delivered_by	文字列	可	—	登録担当者名
registered_at	日時	不可	now()	登録日時
customer_order_no	文字列	可	—	客先注文番号
trace_no	文字列	可	—	発行された TraceNo

インデックス： idx_myou_delivery_logs_lot (lot_id)、 idx_myou_delivery_logs_tenant_company (tenant_id, company_id)

RLS： Tenant Isolation for myou_delivery_logs

3.4 myou_trace_labels（トレーサビリティQR 発行）

列名	型	NULL	既定値	制約・備考
id	UUID	不可	gen_random_uuid()	主キー。公開ページ URL に使用
tenant_id	UUID	不可	current_tenant_id()	テナント識別子
lot_id	UUID	不可	—	外部キー → myou_lots(id)
company_id	UUID	不可	—	外部キー → myou_companies(id)
quantity	整数	不可	1	CHECK (quantity > 0)
expiration_date	日付	不可	—	有効期限
trace_no	文字列	不可	—	TraceNo。テナント内で一意
process_status	文字列	不可	'unused'	CHECK (process_status IN ('unused', 'used', 'alert_ignored'))
created_at	日時	不可	now()	発行日時

インデックス： idx_myou_trace_labels_lot (lot_id)

RLS： Tenant Isolation for myou_trace_labels

3.5 myou_alert_logs（アラート送信履歴）

列名	型	NULL	既定値	制約・備考
id	UUID	不可	gen_random_uuid()	主キー

列名	型	NULL	既定値	制約・備考
tenant_id	UUID	不可	current_tenant_id()	テナント識別子
company_id	UUID	不可	—	外部キー → myou_companies(id) (ON DELETE CASCADE)
sent_at	日時	不可	now()	送信日時
target_trace_nos	文字列配列	不可	—	送信対象の TraceNo 一覧
status	文字列	不可	'success'	success / error
error_message	文字列	可	—	送信失敗時のエラー内容
process_status	文字列	可	—	CHECK (NULL または unused/used/alert_ignored)

インデックス: idx_myou_alert_logs_tenant_sent (tenant_id, sent_at DESC)

RLS: Tenant Isolation for myou_alert_logs

施工会社を削除すると、その会社宛のアラート送信履歴も連鎖削除される (ON DELETE CASCADE)。ただし出荷履歴・QR発行履歴からの参照がある会社は外部キー制約により削除できないため、実運用で履歴が失われることはない。

4. データベース関数（RPC）詳細設計

在庫数量を変更する処理は、複数テーブルへの更新を伴い、かつ同時実行時の不整合が業務影響に直結するため、データベース関数として実装し、行ロックを取得したうえで単一トランザクションで実行する。

いずれの関数も `SECURITY INVOKER`（呼び出し利用者の権限で実行）とし、RLS によるテナント分離を維持する。

4.1 myou_receive_lot（入荷登録）

シグネチャ

```
myou_receive_lot(
  p_lot_no          text,  -- ロット番号
  p_qr_payload      text,  -- ロットQR の内容
  p_expiration_date date,  -- 有効期限
  p_quantity        integer, -- 入荷数量
  p_received_at     date   -- 入荷日
) RETURNS TABLE (lot_id uuid, is_new boolean, previous_status text)
```

処理

順	処理内容
1	自テナント内の <code>lot_no</code> 一致行を <code>FOR UPDATE</code> で検索し、行ロックを取得する
2	該当なしの場合：新規ロットを登録する（ <code>quantity_total</code> = <code>quantity_remaining</code> = 入荷数量、 <code>status</code> = <code>in_stock</code> ）。戻り値 <code>is_new</code> = <code>true</code>
3	該当ありの場合： <code>quantity_total</code> と <code>quantity_remaining</code> に入荷数量を加算し、有効期限・入荷日を更新、 <code>status</code> を <code>in_stock</code> に設定する。戻り値 <code>is_new</code> = <code>false</code> 、 <code>previous_status</code> に更新前の状態を返す

設計意図：同一ロットへ同時に入荷登録が行われた際、行ロックがないと後勝ちで加算が失われる（lost update）。`FOR UPDATE` により直列化する。

4.2 myou_deliver_from_lot（出荷登録）

シグネチャ

```
myou_deliver_from_lot(
  p_lot_no          text,
  p_company_id      uuid,
  p_quantity        integer,
  p_delivered_by    text,
  p_delivery_date   date,
  p_trace_no        text,
  p_customer_order_no text DEFAULT NULL,
  p_trace_label_id  uuid DEFAULT NULL
) RETURNS TABLE (lot_id uuid, expiration_date date)
```

処理

順	処理内容	異常時
1	自テナント内の <code>lot_no</code> 一致行を <code>FOR UPDATE</code> で検索	該当なし：ロット % が見つかりません
2	残数と出荷数量を比較	不足：在庫数量が不足しています (残り%個)
3	<code>p_company_id</code> が自テナントの施工会社か検証	不一致：出荷先（施工会社）が見つかりません
4	<code>quantity_remaining</code> を減算。0 になった場合は <code>status</code> を <code>depleted</code> に、それ以外は <code>in_stock</code> に更新	—
5	<code>myou_delivery_logs</code> に出荷履歴を登録（客先注文番号・TraceNo を含む）	—
6	<code>myou_trace_labels</code> にトレーサビリティQR 発行を登録	—
7	ロットID と有効期限を返す	—

設計意図

- 手順 4～6 を同一トランザクションで実行することにより、「在庫は減ったが履歴が残っていない」といった部分的な反映を防ぐ
- 手順 3 のテナント検証は、本関数が `SECURITY INVOKER` で直接呼び出し可能であることを踏まえ、テナントを越えた出荷登録を防ぐために設けている
- 残数不足時にロットをまたぐ自動引当は行わない。業務上、どのロットから払い出したかが追跡上重要なため、別ロットの再スキャンを利用者に促す

5. サーバー処理詳細設計

5.1 共通事項

項目	仕様
認証確認	全ての読み取り・更新処理の冒頭で <code>getServerUser()</code> を呼び、 <code>tenant_id</code> を取得できない場合は処理を中断する（更新処理は <code>{ success: false, error: '認証エラー' }</code> を返す）
入力検証	更新処理は Zod スキーマで検証し、最初のエラーメッセージを返す
絞り込み	全てのクエリに <code>tenant_id</code> 条件を明示的に付与する（RLS と併せた二重防御）
再描画	更新成功後、関連画面のキャッシュを無効化する（ <code>revalidatePath</code> ）
エラー処理	データベースエラーはサーバー側ログへ詳細を出力し、利用者へは定型メッセージのみ返す

5.2 入力検証スキーマ

スキーマ	対象	検証内容
<code>receiveLotSchema</code>	入荷登録	<code>expiration_date</code> : YYYY-MM-DD 形式かつ実在する日付 / <code>quantity</code> : 1以上100,000以下の整数 / <code>scanned_lot_no</code> : 任意
<code>deliverFromLotSchema</code>	出荷登録	<code>lot_no</code> : 1文字以上 / <code>company_id</code> : UUID 形式 / <code>quantity</code> : 1以上の整数 / <code>customer_order_no</code> : 50文字以内（任意）
<code>upsertCompanySchema</code>	施工会社	<code>name</code> : 1~100文字 / <code>email_address</code> : 255文字以内かつメール形式（空文字は未登録として許容）
<code>companyIdSchema</code>	会社ID	UUID 形式
<code>processStatusSchema</code>	処理ステータス	<code>unused</code> / <code>used</code> / <code>alert_ignored</code> のいずれか

5.3 更新処理（Server Actions）

receiveLot（入荷登録）

項目	内容
入力	<code>{ scanned_lot_no?, expiration_date, quantity }</code>
処理	①認証確認 ②入力検証 ③ロット番号の決定（スキャン値、または当日通番+1で自動採番） ④ <code>myou_receive_lot</code> 呼び出し ⑤画面キャッシュ無効化
自動採番	当日発行分（LOT-YYYYMMDD-%）を取得し、通番の最大値を 数値比較 で求めて+1する
戻り値	<code>{ success, registered_lot_no?, warning?, error? }</code>
警告条件	更新前 <code>depleted</code> → 再入荷警告 / 更新前 <code>in_stock</code> → 数量加算警告

項目	内容
無効化対象	入荷登録画面、在庫一覧画面

deliverFromLot（出荷登録）

項目	内容
入力	{ lot_no, company_id, quantity, customer_order_no? }
処理	①認証確認 ②入力検証 ③施工会社の存在確認と出荷先No取得 ④TraceNo 採番 ⑤発行ID（UUID）の事前採番 ⑥ myou_deliver_from_lot 呼び出し ⑦画面キャッシュ無効化 ⑧QR ペイロード生成
発行ID の事前採番	QR に埋め込む公開ページ URL を組み立てるため、登録前にアプリケーション側で UUID を生成し、RPC へ引き渡す
基準 URL	NEXT_PUBLIC_APP_URL → VERCEL_URL → http://localhost:3000 の優先順で決定する
戻り値	{ success, label?: { trace_no, lot_no, company_no, quantity, expiration_date, qr_payload }, error? }
無効化対象	出荷登録画面、在庫一覧画面、出荷リスト画面

sendManualAlert（アラート送信）

項目	内容
入力	companyId（UUID）
処理	①認証確認 ②会社ID検証 ③対象 QR 発行分の抽出（期限が本日～30日後） ④出荷履歴から使用数を突合し未使用数量を算出 ⑤未使用数量が0の行を除外 ⑥処理ステータスが unused の行のみ抽出 ⑦送信先メールアドレス確認 ⑧メール送信 ⑨送信履歴の記録 ⑩画面キャッシュ無効化
戻り値	{ success } または { success: false, error }
履歴記録	送信成否にかかわらず記録する（失敗時は status='error' とエラー内容を保存）

updateTraceProcessStatus（処理ステータス更新）

項目	内容
入力	labelId（UUID）、processStatus
処理	①認証確認 ②ID・ステータス検証 ③自テナント条件付きで更新 ④画面キャッシュ無効化
画面上の選択肢	「未使用」「アラート無視」の2種類。「使用済」は出荷リストの使用数から判定するため直接選択させない

updateDeliveryUsedQuantity（使用数更新）

項目	内容
入力	logId（UUID）、usedQuantity（整数）
処理	①認証確認 ②ID検証 ③0以上の整数か検証 ④対象出荷履歴の取得 ⑤出荷数量以下か検証 ⑥更新 ⑦画面キャッシュ無効化
エラー	対象なし： 出荷履歴が見つかりません。 / 範囲外： 使用数は出荷数量（N）以下にしてください。

項目	内容
無効化対象	出荷リスト画面、有効期限監視画面

upsertCompany / deleteCompany（施工会社の保守）

項目	内容
新規登録	自テナントの <code>company_no</code> 最大値+1を採番して登録する
更新	<code>id</code> と <code>tenant_id</code> の両方を条件に更新する
メールアドレス	空文字は NULL（未登録）として保存する
削除	<code>id</code> と <code>tenant_id</code> の両方を条件に削除する
削除不可	外部キー違反（エラーコード 23503）を検出し、参照されている旨のメッセージを返す

5.4 読み取り処理（Queries）

関数	取得内容	並び順	上限
<code>getCompanies()</code>	施工会社一覧	会社名の昇順	—
<code>getLots()</code>	残数1以上のロット	有効期限昇順 → ロット番号昇順 → 残数昇順	1,000件
<code>getDeliveryLogs()</code>	出荷履歴（ロット・会社を結合）	出荷日降順 → 登録日時降順	500件
<code>getExpiringTraceLabels()</code>	期限30日以内の QR 発行分	有効期限昇順	—
<code>getAlertLogs()</code>	アラート送信履歴	送信日時降順	50件
<code>getLotTrace(identifier)</code>	ロット情報と出荷履歴	出荷日降順 → 登録日時降順	—
<code>getPublicTraceInfo(id)</code>	公開ページ用の製品情報	—	1件

getLotTrace の検索仕様

順	処理
1	入力値を <code>lot_no</code> として自テナント内を検索
2	該当なしの場合、 <code>trace_no</code> として <code>myou_trace_labels</code> を検索し、紐付く <code>lot_id</code> からロットを取得
3	いずれも該当なしの場合は <code>null</code> を返す
4	ロットが特定できた場合、そのロットの出荷履歴（施工会社名を含む）を取得する

getExpiringTraceLabels の抽出仕様

順	処理
1	有効期限が「本日（日本時間）」～「30日後」の範囲にある QR 発行分を取得
2	同一 <code>TraceNo</code> の出荷履歴から使用数を取得して突合
3	未使用数量 = 出荷数量 - 使用数（下限0）を算出
4	未使用数量が0の行を除外する

getPublicTraceInfo の特例

公開ページはログインを前提としないため、RLS を適用する通常の接続では取得できない。本関数のみ管理者用クライアント（RLS バイパス）を使用する。安全性は次の設計で担保する。

対策	内容
入力の厳格化	引数が UUID 形式に一致しない場合、データベースへ問い合わせず null を返す
取得列の限定	ロット番号・TraceNo・出荷先No・有効期限のみを取得する
単一行の取得	主キー（id）による1件取得に限定し、一覧・検索を提供しない
推測困難性	発行ID は UUID v4 であり、総当たりによる列挙は現実的でない

6. 共通ロジック詳細設計

6.1 QR 解析・採番（lib/qr-parser.ts）

関数	機能
parseLotQrContent(text)	ロットQRを解析し、ロット番号・製造日・有効期限を返す。書式不一致時は全文をロット番号として扱う
parseTraceQrContent(text)	トレーサビリティQRを解析する。URL形式（新書式）とキー・値形式（旧書式）の両方に対応
extractSearchIdentifier(text)	検索用の識別子を取り出す。TraceNoを優先し、無ければロット番号を返す
buildLotQrPayload(lotNo, mfg, exp)	ロットQRの文字列を組み立てる
buildLotNo(dateYmd, sequence)	LOT-YYYYMMDD-NNNNを組み立てる
getMaxLotSequence(lotNos, dateYmd)	当日発行済みロット番号から最大通番を求める
buildTraceNo(dateYmd, sequence)	YYYYMMDD-NNNNを組み立てる
getMaxTraceSequence(traceNos, dateYmd)	当日発行済み TraceNo から最大通番を求める
buildTraceQrPayload(baseUrl, id, traceNo)	公開ページ URL を組み立てる

通番の最大値は、文字列の辞書順では5桁以上（10000～）で大小関係を誤るため、必ず数値へ変換して比較する。

6.2 有効期限計算（lib/expiration.ts）

関数	仕様
getDaysUntilExpiration(expirationDate, now?)	有効期限までの残日数を返す。期限当日は 0、期限切れは負数、未設定・不正日付は null

日本時間基準の計算：日付文字列を必ず YYYY-MM-DDT00:00:00+09:00 として解釈し、「日本時間の本日0時」との差分を暦日で求める。UTC 解釈とローカル解釈が混在すると残日数が±1日ずれるため、明示的にタイムゾーンを指定している。

画面表示への反映

残日数	表示	色
null	期限未設定	灰
0以下	期限切れ	赤
1～30	残りN日	黄
31以上	残りN日	緑

6.3 処理ステータス（lib/process-status.ts）

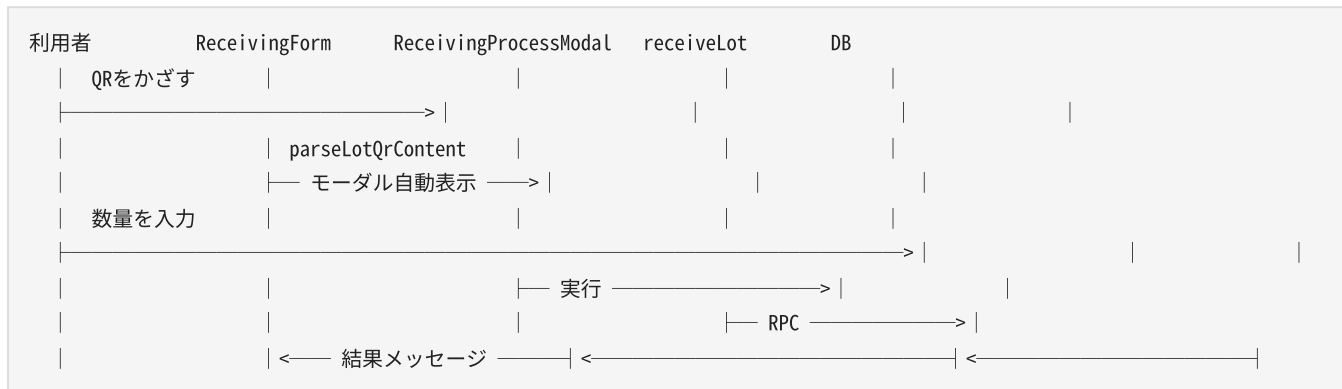
関数・定数	仕様
PROCESS_STATUS_VALUES	['unused', 'used', 'alert_ignored']
PROCESS_STATUS_EDIT_VALUES	画面で選択可能な値: ['unused', 'alert_ignored']
processStatusLabel(status)	表示名を返す (未使用 / 使用済 / アラート無視)
filterUnusedForAlert(rows)	アラート送信対象 (unused) のみを抽出する

7. 画面詳細設計

7.1 入荷登録（/myou/receiving-scan）

構成：ReceivingForm（スキャナ制御）＋ QrScanner（カメラ）＋ ReceivingProcessModal（数量入力）

処理シーケンス



メッセージ

種別	メッセージ	意味
成功	入荷登録成功: {ロット番号}	新規ロットとして登録された
警告	{ロット番号} は既に入荷済みです (数量を追加登録しました)。	既存ロットへ数量を加算した
警告	{ロット番号} は出荷済み (残数0) でしたが、再入荷として在庫に戻しました。	返品等による再入荷
エラー	認証エラー	再ログインが必要
エラー	ロット番号の採番に失敗しました。	採番用の照会に失敗
エラー	入荷登録に失敗しました。もう一度お試しください。	登録処理の失敗

QRを読み取れない場合：「入荷処理へ進む」ボタンからモーダルを直接開き、ロット番号を自動採番して登録できる。

7.2 在庫一覧（/myou/inventory）

項目	仕様
見出し	件数と残数合計を表示（例：「入荷済み・未出荷のロット（12件、残数計 340本）」）
表示列	ロット番号／残数／入荷総数／有効期限／入荷日／期限状況
並べ替え	ロット番号・残数・有効期限・入荷日
検索	ロット番号による絞り込み
空表示	「現在、在庫はありません。」

7.3 出荷登録（/myou/delivery-scan）

構成：DeliveryForm（共通入力＋タブ制御）＋ QrScanner / InventoryDeliveryTable ＋ DeliverQuantityModal ＋ TraceQrModal

共通入力

項目	必須	検証
出荷先（施工会社）	必須	未選択時はスキャン・出荷操作を受け付けない
受注数量	必須	1以上の整数
客先注文番号	任意	50文字以内

引当方法

タブ	操作
QRスキャン	ロットQRを読み取り、共通入力の数量で即座に出荷登録する
在庫表より（既定）	在庫一覧から対象ロットの「出荷」ボタンを押し、モーダルで数量を確認して登録する

登録成功時：トレーサビリティQR印刷モーダル（TraceQrModal）が自動的に開く。

項目	仕様
表示内容	ロット番号、TraceNo、出荷数量、有効期限、QR画像
印刷部数	既定値は出荷数量。上限1,000部
印刷方式	同一内容のQRを指定部数だけ複製し、ブラウザの印刷機能で出力する

メッセージ

種別	メッセージ
成功	出荷登録成功: {ロット番号} ({数量}個)
エラー	先に出荷先（施工会社）を選択してください。
エラー	受注数量を入力してください。
エラー	ロット {ロット番号} が見つかりませんでした。
エラー	在庫数量が不足しています（残りN個）。別のロットをスキャンしてください。
エラー	出荷先（施工会社）が見つかりませんでした。
エラー	TraceNoの採番に失敗しました。
エラー	出荷登録に失敗しました。もう一度お試しください。

7.4 出荷リスト（/myou/delivery-history）

列	内容
出荷日	出荷登録日
ロット番号	引当元ロット
トレースNo	発行された TraceNo（旧データは -）
得意先No	施工会社の出荷先No
出荷先	施工会社名
客先注文番号	未入力時は -

列	内容
出荷数量	出荷本数
使用数	施工会社での使用実績
期限状況	期限切れ／残りN日／期限未設定。出荷数量と使用数が一致する行は -
編集	使用数編集モジュールを開く

出荷数量と使用数が一致する行は、全量が使用済みで期限管理の対象外となるため、期限状況を表示しない。

7.5 トレーサビリティ検索（/myou/traceability）

項目	仕様
入力方法	①ロット番号／TraceNo の手入力 ②QR スキャン（読み取り後に自動照会）
表示（ロット）	ロット番号、現在の状態、残数／入荷総数、有効期限（期限切れは赤）、入荷日
表示（出荷履歴）	出荷先、出荷日、数量、TraceNo、客先注文番号、登録担当者（出荷日の新しい順）
該当なし	「製品が見つかりません」
エラー	「照会中にエラーが発生しました。時間を置いて再度お試しください。」（検索条件は保持する）

7.6 有効期限監視・アラート（/myou/expiration-alerts）

区分	仕様
ヘッダー	アラート対象件数、直近送信日時
グループ化	施工会社別。施工会社情報が欠落した行は表示対象外
明細	TraceNo、ロット番号、出荷数量、使用数、未使用数量、有効期限、残り日数、処理ステータス
送信ボタン	施工会社ごとに配置。メールアドレス未登録の場合は無効
処理ステータス編集	明細の鉛筆アイコンから「未使用」「アラート無視」を選択
送信履歴	送信日時、送信先、対象 TraceNo、成否、エラー内容（直近50件）

メッセージ

種別	メッセージ	対処
成功	アラートメールを送信しました。	送信履歴で確認
エラー	送信先のメールアドレスが登録されていません。	納入先登録で設定する
エラー	対象の期限間近製品が見つかりませんでした。	画面を再読み込みする
エラー	送信対象の未使用製品がありません。	処理ステータスを確認する
エラー	処理ステータスの更新に失敗しました。	再試行する

7.7 納入先登録（/myou/companies）

項目	仕様
一覧	出荷先No、会社名、メールアドレス、登録日
新規登録	モーダルで会社名（必須）・メールアドレス（任意）を入力
編集	一覧の鉛筆アイコン
削除	一覧のゴミ箱アイコン（確認ダイアログあり）

メッセージ

種別	メッセージ
成功	新規登録が完了しました。 / 更新が完了しました。 / 削除が完了しました。
エラー	会社名を入力してください。
エラー	メールアドレスの形式が正しくありません。
エラー	この会社は出荷履歴またはトレーサビリティQR発行履歴から参照されているため削除できません。
エラー	出荷先Noの採番に失敗しました。

7.8 製品情報公開ページ（ /p/myou/trace/{id} ）

項目	仕様
認証	不要
表示項目	ロット番号、施工先No、トレースNo、有効期限
販売元表示	ミュー株式会社および公式サイトへのリンク
該当なし	「見つかりません」 ページ（404）
設計方針	表示内容は URL に含めず、発行ID からデータベース参照して表示する

8. メール送信詳細設計

項目	仕様
実装	Nodemailer による SMTP 送信
送信元	環境変数 MAIL_FROM (既定値: "製品管理" <noreply@example.com>)
接続先	環境変数 SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASS
開発環境	SMTP_USER 未設定時はローカルのメール確認用サーバーへ送信する (本番へ誤送信しないため)
件名	【重要】製品有効期限に関するお知らせ ({会社名}様)
本文形式	プレーンテキスト
エラー処理	送信失敗時も例外を送出せず、成否とエラー内容を戻り値で返し、履歴に記録する

本文構成

{会社名} 担当者 様 いつもお世話になっております。 現在ご使用いただいている製品の中に、有効期限が間近 (30日以内) のものが含まれています。安全のため、期限内にご使用いただくか、適切な処置をお願い申し上げます。 ■ 対象製品リスト ロット番号: {ロット番号} / 数量: {未使用数量}個 / 有効期限: {有効期限} (TraceNo: {TraceNo}) … (対象件数分) ご不明な点がございましたら、管理者までお問い合わせください。

本文に記載する数量は、出荷数量ではなく **未使用数量** (出荷数量 - 使用数) である。

9. エラーハンドリング設計

9.1 方針

No	方針
EH-01	利用者へ返すメッセージには、内部構造・SQL・スタックトレースを含めない
EH-02	原因究明に必要な詳細（エラーコード、ヒント）はサーバー側ログにのみ出力する
EH-03	業務上の判断が必要な事象（重複入荷、再入荷など）は、エラーではなく警告として処理を継続する
EH-04	照会処理の失敗は画面遷移させず、検索条件を保持したままエラーバナーを表示する
EH-05	読み取り処理でエラーが発生した場合は空配列を返し、画面全体の停止を避ける

9.2 エラー区分

区分	表示	例
成功	緑バナー	登録完了
警告	黄バナー	重複入荷、再入荷
エラー（入力）	赤バナー	必須項目未入力、書式不正、在庫不足
エラー（システム）	赤バナー（定型文）	データベース接続失敗、メール送信失敗
画面エラー	error.tsx による再試行画面	予期しない例外

10. 単体テスト

lib/ 配下の純粋関数を対象に単体テストを実装している。

テスト対象	主なテスト観点
qr-parser.test.ts	ロットQR／トレースQRの解析（新旧両書式）、書式不一致時のフォールバック、通番の抽出と最大値算出（5桁以上を含む）、各種文字列の組み立て
expiration.test.ts	残日数の計算（当日＝0、期限切れ＝負数、未設定＝null）、日本時間基準での境界判定
process-status.test.ts	表示名の対応、アラート送信対象（未使用のみ）の抽出

11. 制限値一覧

項目	値	定義箇所
入荷数量の上限	100,000	<code>receiveLotSchema</code>
客先注文番号の桁数	50文字	<code>deliverFromLotSchema</code>
会社名の桁数	100文字	<code>upsertCompanySchema</code>
メールアドレスの桁数	255文字	<code>upsertCompanySchema</code>
在庫一覧の取得件数	1,000件	<code>LOTS_FETCH_LIMIT</code>
出荷リストの取得件数	500件	<code>DELIVERY_LOGS_FETCH_LIMIT</code>
アラート送信履歴の表示件数	50件	<code>getAlertLogs()</code>
有効期限アラートの対象日数	30日	<code>EXPIRATION_ALERT_DAYS</code>
トレーサビリティQRの印刷部数上限	1,000部	<code>MAX_PRINT_COPIES</code>
ファイルアップロードの上限	50MB	<code>next.config.ts</code> (基盤共通)

12. 環境変数

変数名	用途	必須
NEXT_PUBLIC_SUPABASE_URL	データベース API の接続先	必須
NEXT_PUBLIC_SUPABASE_ANON_KEY	一般利用者用の公開キー	必須
SUPABASE_SERVICE_ROLE_KEY	サーバー専用キー（公開ページの参照に使用）	必須
NEXT_PUBLIC_APP_URL	トレーサビリティQR に埋め込む基準 URL	推奨
SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASS	メール送信設定	アラート利用時は必須
MAIL_FROM	送信元アドレス	任意

SUPABASE_SERVICE_ROLE_KEY はデータベースのアクセス制御を迂回できる資格情報である。サーバー側のみで使用し、ブラウザへ露出させてはならない。